React project through local environment

Before you start, make sure that you have already install NodeJS, npm and Typescript in your local environment.

You can launch a react project under your development folder (you can declare where the folder is). Then you can execute the command as below:

npm init vite@latest [project name]

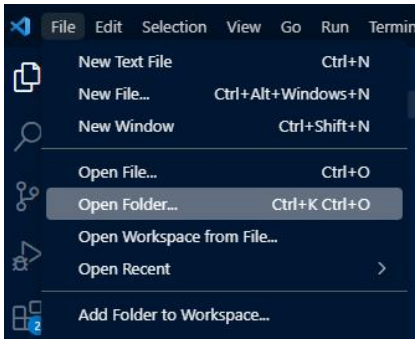
Answer the question as below:

```
√ Package name: ... tutorial4
√ Select a framework: » React
√ Select a variant: » TypeScript + SWC

Scaffolding project in C:\Users\hcwong\Development\Tutorial4...

Done. Now run:

  cd Tutorial4
  npm install
  npm run dev
```

SWC = Spectrum Web Components (SWC) is a collection of wrapper components works like native React components in a React application. As these components serve as a bridge between React and SWC, their properties and event names are directly derived from the corresponding SWC components.

Using Visual Studio Code

Open the folder in Visual Studio Code by File > Open Folder



Once you have opened the folder, you can execute the following command in the terminal (make sure that you are executing the terminal using PowerShell).

npm run dev

VT6003CEM Web API Development (Hong Kong version)
Copyright to Vocational Training Council.
This document is for VTC students for their own use in completing their learning for this module and should not be passed to third parties or posted on any website. Any infringements of this rule should be reported to cw-it@vtc.edu.hk

And clicking on the local link (Ctrl-Click) to launch the page.

Creating a Blog Home Page

Understanding Components, Props, State and Events in React

In this worksheet we will begin creating a front page for our blog using React and a CSS component library called Ant-Design:

- https://www.freecodecamp.org/news/typescript-for-react-developers
- https://ant.design/docs/react/introduce

The main objective is to get you familiar with the idea of React components and how they manage their data: props and state. "Useful" functionality will be developed after this week.

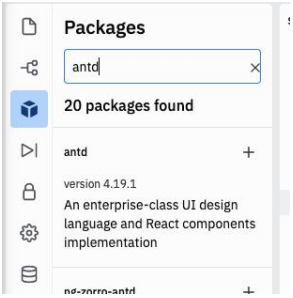
We use Ant Design simply to make the components look nice and avoid having to re-code common components by hand, such as form elements, buttons, modals, and so on. Other options that do the same as Ant Design are:

- https://react-bootstrap.github.io/
- https://material-ui.com/

Add Ant Design to the project

We need to install Ant Design. As always, we use NPM to do the work with the command:
npm install antd

or through replit packages platform (as shown on the right)



Use some Ant Design components

To make sure Ant Design works properly let's render something.

Open the **App.tsx** file and make sure you already removed all template code, and this is the only thing you have **<>** tag within the return() *(change from **<main>** to **<>** if you are working in replit)*

Now, add the following script within < > and </ > tag:

```
    <div >
      <Card title="Default card" style={{ width: 300 }}>
        <p>Card content</p>
        <p>Card content</p>
```

VT6003CEM Web API Development (Hong Kong version)
Copyright to Vocational Training Council.
This document is for VTC students for their own use in completing their learning for this module and should not be passed to third parties or posted on any website. Any infringements of this rule should be reported to cw-it@vtc.edu.hk

```
        <p>Card content 123</p>
      </Card>

      <br/>

      <Button type="primary" onClick={onClick}>Button</Button>
      <Button type="primary" danger>Button</Button>

      <br/>

      <DatePicker onChange={onChange} />
    </div>
```

We have used some components from Ant-Design so let's import them, on the top of the page add the following:

```
import { Card, Button, DatePicker } from 'antd';
```

You will also have noticed that we included two JavaScript handler functions: 'onClick' in one of the Buttons, and 'onChange' in the DatePicker. To show that we can react to simple events, add these handler functions above the App() function in the file:

```
let counter = 0

const onChange: DatePickerProps['onChange'] = (date, dateString) => {
  console.log(date, dateString);
};

const onClick = (event: any) => {
  console.log(counter++)
}
```

(Note: if you are using Visual Studio Code, the new required function will automatically import at the beginning of the code)

Finally, we need to import ant-design styles into our app, so all components will have their default styles applied to them. To do that, open **App.tsx**, select everything and delete the existing code.

At the top of the file add the following import statement:

```
import 'antd/dist/reset.css';
```

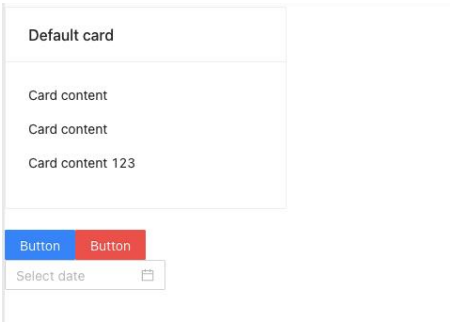
Test the React app

To launch the development server for the React app simply click the run button (or the following command from inside the local directory.

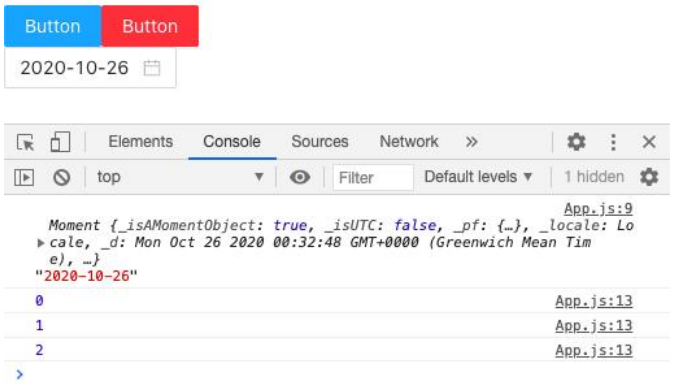
npm run dev

NB: once it is running you do not need to stop the server and restart, since Create-React-App has already installed a tool for you that detects when the code is changed and restarts the server.

Copyright to Vocational Training Council.
This document is for VTC students for their own use in completing their learning for this module and should not be passed to third parties or posted on any website. Any infringements of this rule should be reported to cw-it@vtc.edu.hk



To check the event handlers work, open your browser's developer tools and view the JS console (F12). Select a date from the **DatePicker** component and click the blue button a few times. Your console should show the logged output:



Everything is working as expected now.

What we have done so far?

We have added components called **Card**, **Button**, and **DatePicker** from the Ant Design UI library.

Each works in a similar way. For example, the tag **<Card>** is the actual component which itself can have child components in it, such as the **<p>** tags here.

The component **<Card>** has been set with initial properties or parameters

```
title="Card title" style={{ width: 300 }}
```

these properties or parameters are sent to the component when it starts. In React these are the **props** of the component.

TSX markup in TS code

But what are these HTML-like tags such as **<Card>** doing in the middle of JavaScript code anyway?

This syntax is called **TSX**. It is a special syntax used in React, which is compiled by a tool called Babel to actual HTML code. Babel is a tool we would otherwise need to install, but the create-react-app script installed it for us.

VT6003CEM Web API Development (Hong Kong version)
Copyright to Vocational Training Council.
This document is for VTC students for their own use in completing their learning for this module and should not be passed to third parties or posted on any website. Any infringements of this rule should be reported to cw-it@vtc.edu.hk

TSX is very similar to HTML, so if you are familiar with HTML you can start writing and understanding TSX, plus you can write your own TSX code using your HTML knowledge. In fact, all HTML tags you know can be used as they are part of TSX.

However, there are few differences between TSX and HTML you should be aware of:

- `className` attribute instead of `class`
- `htmlFor` attribute instead of `for`
- `TSX uses camelCase for attribute names, for example tabindex becomes tabIndex`
- `<textarea value="something">` instead of `<textarea>something</textarea>`
    - In other words, the text in a text area is actually the value of a prop called 'value' in TSX.
- `<select value="something"><option value="something"></select>` instead of `<select><option selected value="something"></select>`
    - Note that the value prop is added to the select tag itself.
- Tags must be closed, and all tags can be self-closing.

You can embed TypeScript code inside TSX by enclosing it with curly braces {…}, we will be doing this often.

Your first custom React component

So, now you know now what TSX is.

Next, you need to understand components in React and in particular how to build your own. The basic concepts behind components are:

- Components are either **TS classes** or **TS functions.**
- You usually define them and export them from module files, just like any other JS object that you need to re-use in other scripts.
- Components generally return some TSX code. *Remember that anything you would like to render in React should be done with TSX syntax.*
- After you import your component(s), either in to your top-level App() component or in to some other 'container' component, you use them as new a TSX tag using the name you exported.

Let's create our first component which is going to be a simple **hello world** component.

Classes vs Functions

As mentioned above a component can be a class or just a function. If your component only renders something and does not have much internal logic, then you should define it using a function.

Hello component

In this example, our component is simple so we will use a functional component.

Inside the app's **src** folder, create a new folder called **components**

Inside the **components** folder create a file called '**Hello.tsx**'

Copyright to Vocational Training Council.
This document is for VTC students for their own use in completing their learning for this module and should not be passed to third parties or posted on any website. Any infringements of this rule should be reported to cw-it@vtc.edu.hk

Inside **Hello.tsx** file write the following code:

```
import React from 'react';

const Hello = () => {
  const greeting = 'Hello Component!';
  return <h1>{greeting}</h1>;
}
export default Hello;
```

Note it is mandatory to import React for each react component as you see in line 1 here.

Now to use this component in our app, go to **App.tsx**

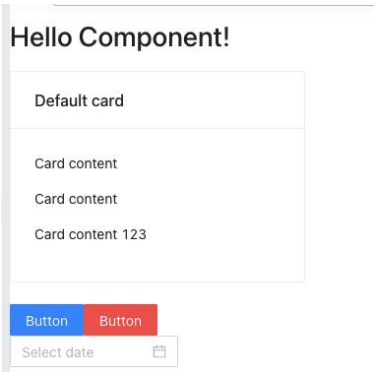
Import the component by adding:

```
import Hello from './components/hello';
```

Then above the card component in the previous **App.tsx** code add the following line:

```
<Hello />
```

And checking the preview again you should see the message now printed.



Sending props to components

To make our Hello component more useful, we can send a parameter/property to it (a prop). The component needs to be modified to be able to receive a prop as follows:

```
import React from 'react';

const Hello = (props) => {
  //we expect props to have a property called name
  const greeting = `Hello ${props.name}`;
  return <h1>{greeting}</h1>;
}

export default Hello;
```

now inside **App.tsx** where we have used our hello component, we need to pass a value to the prop:

VT6003CEM Web API Development (Hong Kong version)
Copyright to Vocational Training Council.
This document is for VTC students for their own use in completing their learning for this module and should not be passed to third parties or posted on any website. Any infringements of this rule should be reported to cw-it@vtc.edu.hk

```
<Hello name="Web API Development"/>
```

You should see the message in the preview now says:

"Hello Web API Development"

⇨ ***Task**: using your knowledge of TypeScript so far, can you adjust the Hello component in a way that if it receives no name it will print a default message 'Hello World'*

State and Events

If we would like to make the Hello component interactive, for example, to turn the name green when the mouse hovers above the name, or grey out when another component is clicked, it will require the component to do one or more of the following:

- respond to (TypeScript initiated) events
- interact with the user
- maintain internal state

To make a new component similar to the one above but with the ability to store 'state', we need to import **useState** from react. Let's create a new file '**components/Goodbye.tsx**'.

```
import React, {useState} from 'react';

const Goodbye = (props) => {

   const [txtColor, setTxtColor] = useState("blue");

   return (
     <>
      <h1>Goodbye {props.name}</h1>
      <h2>Current colour is: {txtColor}</h2>
     </>
   );
}

export default Goodbye;
```

This component can be used in other components exactly like our functional one: you need to import it in **App.tsx** and include the component tag in the appropriate place in the TSX returned from there.

- **Task:** Do this now to include the Goodbye component in your app, and preview the output.
    - Send the component's prop 'name' the value 'everyone'.

Differences between class and function components

As you see there are a few differences in how the class-based component is defined.

Copyright to Vocational Training Council.
This document is for VTC students for their own use in completing their learning for this module and should not be passed to third parties or posted on any website. Any infringements of this rule should be reported to cw-it@vtc.edu.hk

The first thing is that we have a get-set pair **useState** object **txtColor**. This state object is a special object inside React components that you can use to store the internal state values. There is one important rule about this object:

- it should never be mutated, or changed directly
- the only way you can change it after you initialize it is by using **setTxtColor()** method.

We have created a state variable, **txtColor**, so we can track the value. Initially the **txtColor** is set to be blue.

We wish to apply the color state to the Hello message. To do that we will apply a style to the **<h1>** tag so it becomes like this:

```
return <h1 style={{color:txtColor}}>Goodbye {props.name}</h1>;
```

Note the syntax in this block of code. The curly braces { } are used in TSX to identify TS code that you wish to be evaluated. Thus, in the block `{props.name}` the React library will place the value of the name passed as a prop. Contrast this with the block `{{color:txtColor}}`. The double braces indicate we want the TSX to evaluate the TS expression `{color:txtColor}`, which is an **object** in JS. When React evaluates the object it uses the key-value pairs defined in the JS object as the key-value pairs to apply to the corresponding HTML tag attribute.

Simple event handling

Now let's add a mouse event. To do so we will add two functions, **changeColor** and **revertColor** and bind them to the **onMouseEnter** and **onMouseLeave** events on the <h1> tag.

Add the following code:

```
  const changeColor = () => {  setTxtColor("red");  }
  const revertColor = () =>{  setTxtColor("blue");  }
```

Note that we used **setTxtColor()** function to alter the **txtColor** state, we did **not** alter **txtColor** directly. We also pass in an object as an argument containing the name of the property or properties we would like to change. React uses this object by *merging its properties* with those of the existing state object.

The next step is to call these functions with mouse events, so now we set the functions to be called when the events happen using **onMouseEnter** and **onMouseLeave** attributes.

Now return should change as follows:

```
    return(<> <h1 onMouseEnter={changeColor} onMouseLeave={revertColor}
              style={{color:txtColor}}>Goodbye {props.name}</h1></>);
```

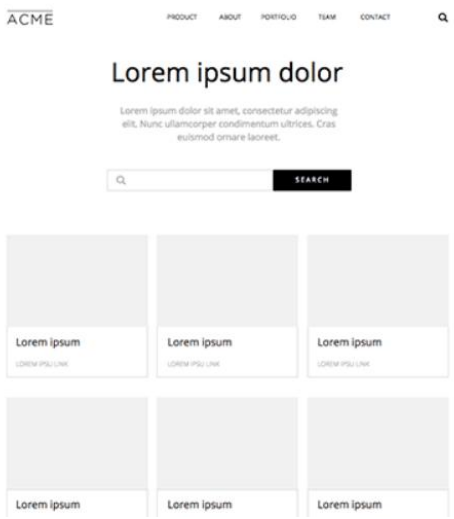

Copyright to Vocational Training Council.
This document is for VTC students for their own use in completing their learning for this module and should not be passed to third parties or posted on any website. Any infringements of this rule should be reported to cw-it@vtc.edu.hk

Basic Worksheet Tasks

As you now understand the concept of components and props you can continue with building a home page for the blog.

Blog home page:

The design of the home page should be something like the following:



- ⇨ As a first step make the card you added render an image, with a title and some description text. To do that consult ant-design documentation for the card at this link.
- ⇨ Try to customise the card so it produces something like this result:

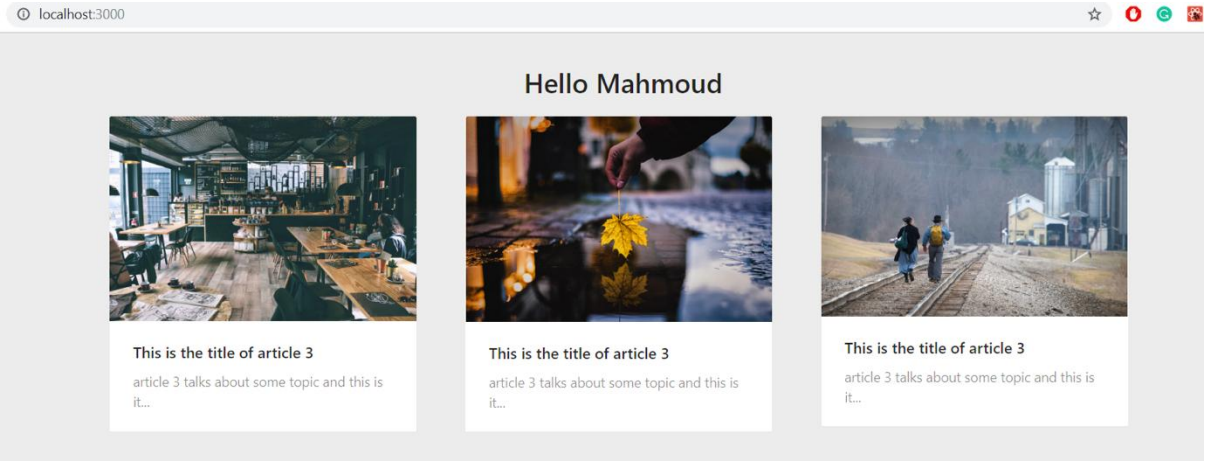
Now you should add multiple cards and customise each one with a different image, title and caption. However, if you try to put more cards in the app directly, they will stack on top of each other.

- ⇨ To solve this issue, we will need to use ant-design's grid which you should read about here.
- ⇨ Use the following row and col settings for the grid (3 cards per row):

Copyright to Vocational Training Council.
This document is for VTC students for their own use in completing their learning for this module and should not be passed to third parties or posted on any website. Any infringements of this rule should be reported to cw-it@vtc.edu.hk

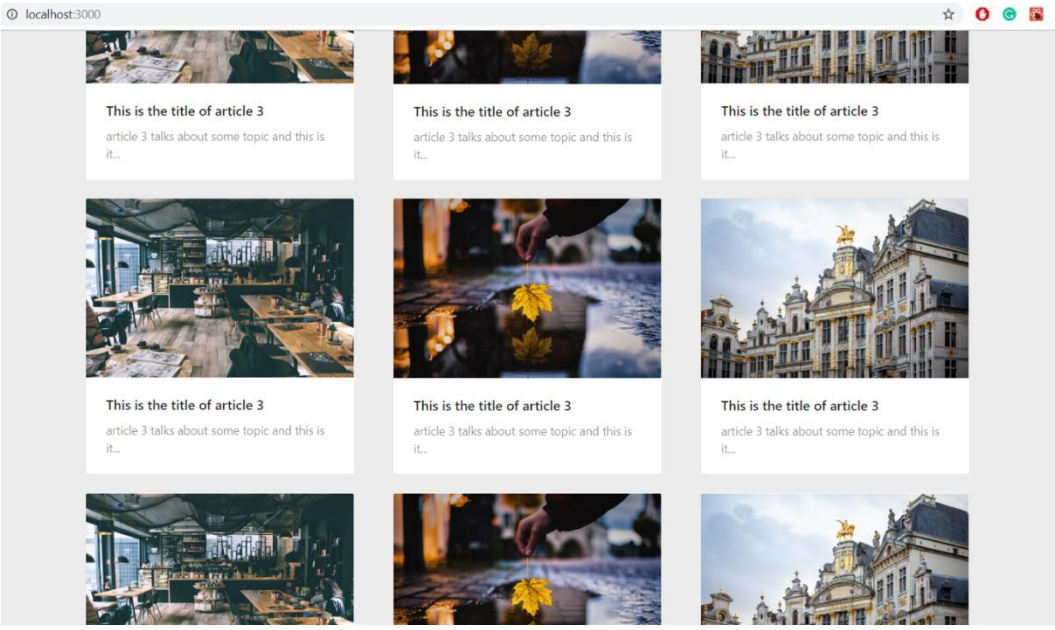
```
<>
<Row type="flex" justify="space-around">
   <Col span={6}>

   </Col>
   <Col span={6}>

   </Col>
   <Col span={6}>

   </Col>
</Row>
</>
```

⇨ Inside each of the <Col> components render one <Card> to give:



⇨ To add more rows simply repeat the code few times and customise the card contents each time to get to the final result.

This is the final result:



VT6003CEM Web API Development (Hong Kong version)
Copyright to Vocational Training Council.
This document is for VTC students for their own use in completing their learning for this module and should not be passed to third parties or posted on any website. Any infringements of this rule should be reported to cw-it@vtc.edu.hk

Advanced Worksheet Tasks

- ⇨ Based on the earlier wireframe, can you improve the Blog Home page and create a complete wireframe using React and Ant-Design components?
  - This includes the header 'nav-bar' links, block title, search form, etc.
  - See https://ant.design/components/overview/ for details on each of the available components
  - At this stage, the wireframe contents should be 'static'. Just use fake data hard coded in the component file(s). We will be retrieving data dynamically in future weeks.

React Routing

The idea of routing in a component-based single-page application is that a 'high-level component' manages the conditional rendering of subtrees of the DOM. These subtrees correspond to the 'pages' or parts of pages that we wish to show at different 'links'.

If we wish to map a specific URL to a specific view/page in our React app we need to manipulate the browser History object and push new states to it (see the links on the History API below). However, doing this manually is time-consuming.

https://developer.mozilla.org/en-US/docs/Web/API/History_API

https://developer.mozilla.org/en-US/docs/Web/API/History_API/Working_with_the_History_API

Review the links above to ensure you understand the methods available in the browser, that enable you to change and manipulate the browser's history when required: ***pushState()*** *and* ***replaceState()****.*

As navigation is such a standard feature in SPAs there are several libraries available to manage this behaviour on our behalf.

We will use React Router to handle all the "high-level" conditional rendering operations for us. This will provide abstractions for "linking to other pages", and "changing the address" of the page being viewed in a consistent way.

React Router Quick-start

Begin by installing the routing library we wish to use as a dependency:

```
$ npm install react-router-dom
```

Warning: The following laboratory is based on the latest version of `react-router-dom`. The following library is based on `react-router-dom version 6+`.

There are some key terms you should know to get started with the routing configuration.

`BrowserRouter`

`BrowserRouter` is the router implementation that uses the HTML5 history API to keep your UI up to date with the browser URL. It is **BrowserRouter**'s responsibility to store all the components and its routes as an object.

Copyright to Vocational Training Council.
This document is for VTC students for their own use in completing their learning for this module and should not be passed to third parties or posted on any website. Any infringements of this rule should be reported to cw-it@vtc.edu.hk

`Routes`

`Routes` components are used to render the default components once the app rendered, and it will switch between routes as needed.

*Note: In react-router-dom version <5, this component name **Switch** and the syntax within this component is different from version 6.*

`Route`

The route is a statement that holds the specific path of the app along with the component's name and renders it once it matches the URL.

`Link`

The link is similar to the HREF link, which allows you to redirect to specific components based on the specified path.

Implementing Default Route Redirect

So far in this guide, you have learned how to install the package and the terms that you are going to use for implementing the routing.

You can achieve the router configuration by using an API called **`BrowserRouter`**, which wraps all the components that reside in our React app like this.:

```
<Router>
    <Routes>
        <Route path="/path1" component={comp1} />
        <Route path="/path2" component={comp2} />
        <Route path="/path3" component={comp3} />
        <Route index component={comp4} />
    </Routes>
</Router>
```

***index** is the keyword that set the route as the default path to display.*

In the above example, the **<Router>** is a parent component that wraps other child components as a **<Route>** that is connected to the specific components.

Let's look at the following example. You are advised to create a new project.

Create sample components.

In the components folder, create 3 files: **`About.tsx`**, **`Dashboard.tsx`** and **`Home.tsx`**. The sample skeleton for these components is as follows:

```
import React from "react";
const Home = () => {
  return <p>Home</p>;
}
export default Home;
```

Remember to update the italic name above to match the component name.

VT6003CEM Web API Development (Hong Kong version)
Copyright to Vocational Training Council.
This document is for VTC students for their own use in completing their learning for this module and should not be passed to third parties or posted on any website. Any infringements of this rule should be reported to cw-it@vtc.edu.hk

Looking at **`App.tsx`**, update the code as follow

```
import { Layout, Space } from 'antd';
import React from 'react';
import './App.css';
import { BrowserRouter as Router,
        Routes, Route, Link
       } from 'react-router-dom'

import Home from './components/home'
import Dashboard from './components/dashboard'
import About from './components/about'

const { Header, Content, Footer } = Layout;

const App = () => {
  return (
    <Router>
      <Header>
        <nav>
          <Space>
            <Link to="/">Home</Link>
            <Link to="/dashboard">Dashboard</Link>
            <Link to="/about">About</Link>
          </Space>
        </nav>
      </Header>
      <Content>
        <Routes>
          <Route index element={ <Home />} />
          <Route path="/about" element={ <About />} />
          <Route path="/dashboard" element={ <Dashboard /> } />
        </Routes>
      </Content>
      <Footer>
        <p>VT6003CEM Demo</p>
      </Footer>

    </Router>
  );
}

export default App;
```

By using Ant Design Layout, you can clearly define the different areas of the application.

Now you can test and see the change in the SPA.

Copyright to Vocational Training Council.
This document is for VTC students for their own use in completing their learning for this module and should not be passed to third parties or posted on any website. Any infringements of this rule should be reported to cw-it@vtc.edu.hk

Read JSON data

Before we connect to the API to retrieve the data, let's try to get the data from the local JSON file. You may create a demo JSON (**articles.json**) in the data folder within the **components** folder. The sample content is as below:

```
[
  {"id": 1, "title":"hello article", "fullText":"some text here to fill
the body"},
  {"id": 2, "title":"another article", "fullText":"again here is some text
here to fill"},
  {"id": 3, "title":"coventry university", "fullText":"some news about
coventry university"},
  {"id": 4, "title":"smart campus", "fullText":"smart campus is coming to
IVE"}
]
```

We will display all the articles at the landing page. In order not to make the **Home.tsx** too complex, create a new component **Articles.tsx** as below:

```
import React from 'react';
import { Link } from 'react-router-dom';
import { Card, Col, Row } from 'antd';
import articles from './data/articles.json'

const Article = () => {
  return (
    <Row justify="space-around">
    {
      articles &&
        articles.map(({id, title, fullText})=> (
          <Col span={8} key={id} >
            <Card title={title} style={{ width: 300 }} >
              <p>{fullText}</p>
              <p></p>
              <Link to={`/a/${id}`}>Details</Link>
            </Card>
          </Col>
      ))
    }
    </Row>);
}
export default Article;
```

We try to make use of the **Card** component and fill in the data one by one. Remind to update **Home.tsx** to map to the **Article.tsx** and see the result.

VT6003CEM Web API Development (Hong Kong version)
Copyright to Vocational Training Council.
This document is for VTC students for their own use in completing their learning for this module and should not be passed to third parties or posted on any website. Any infringements of this rule should be reported to cw-it@vtc.edu.hk

Dynamic mapping data by passing properties

Register a new path at the **App.tsx** with "**/a/:aid**" where "**:aid**" is a parameter for some articles id.

Create a new component name **DetailArticle.tsx** to display ONE article dynamically by using properties as below:

```
import React from 'react';
import articles from './data/articles.json';
import { useParams, useNavigate } from 'react-router-dom';
import { Button } from 'antd';
import { RollbackOutlined } from '@ant-design/icons';

const DetailArticle = (props) => {
  const { aid } = useParams();
  const navigate = useNavigate();
  for(const article of articles) {
    if(article.id==aid)  {
      return(
        <>
          <h1>{article.title}</h1>
          <p>{article.fullText}</p>
          <Button type="primary" icon={<RollbackOutlined />}
onClick={()=>navigate(-1)} />
        </>
      );
    }
  }
}
export default DetailArticle;
```

Here, we have created Two items:

1. Each article at "/a/:id" will have a different ID and the post-view component needs to be aware of what it is. To achieve this the **`react-router-dom`** library defines a React hook called '`useParams()`' which we can use.
2. There is a back button with `useNavigate()` which can return to the previous page. (Note: in the previous version, the framework is named useHistory().)

Copyright to Vocational Training Council.
This document is for VTC students for their own use in completing their learning for this module and should not be passed to third parties or posted on any website. Any infringements of this rule should be reported to cw-it@vtc.edu.hk

Worksheet Tasks

You now have the basis for adding navigation to various pages in your SPA.

- → Add a 'Register' and a 'Login' link to the nav bar and define placeholder pages for them in the 'src/components' folder. Update your router in 'App' to render the appropriate page when the given link is clicked.
- → Think about and write down which specific pages you will need in your coursework SPA. Think of the required linking structure that you will need to set up with React Router (or a similar library) and begin implementing placeholder pages and links to them in your app.

VT6003CEM Web API Development (Hong Kong version)
Copyright to Vocational Training Council.
This document is for VTC students for their own use in completing their learning for this module and should not be passed to third parties or posted on any website. Any infringements of this rule should be reported to cw-it@vtc.edu.hk